

学习效果评估和巩固智能体

本项目演示视频百度网盘地址为

链接: <https://pan.baidu.com/s/1Ava-pZd-m1VnVztiq0SIHQ?pwd=1yy3> 提取码: 1yy3

目录

学习效果评估和巩固智能体

目录

一、项目概览

- 项目背景
- 架构设计
 - 整体架构设计
 - 部署层
 - 展示层
 - 应用层
 - 存储层
 - 云原生技术栈选型与决策 (Tech Stack Selection & Rationale)
 - 2.2.1 存储架构: 高可用与数据安全
 - 2.2.2 应用架构: 微服务解耦
 - 2.2.3 容器化构建: 多阶段构建 (Multi-stage Build)
 - 核心业务数据流设计 (Core Data Flow Design)
 - 2.3.1 多模态资料摄入链路
 - 2.3.2 考试状态同步链路
 - 高并发与稳定性优化策略 (Optimization Strategy)
 - 2.4.1 资源调度与隔离 (Resource Isolation)
 - 2.4.2 性能优化 (Performance Tuning)

二、核心技术实现

- 团队分工总览 (组内均分)
- 架构实现 (成员3)
 - 存储架构高可用升级
 - 前端交互与可视化深度优化
 - 容器化运维与资源治理
- 算法实现 (成员1 & 成员2)
 - 用户注册与登录 (成员1)
 - 上传资料 (成员1 & 成员2)
 - 上传流程
 - 单个pdf文件上传 (成员1)
 - 多个pdf上传 (成员2)
 - 录音上传 (成员2)
 - 扫描件上传 (成员1)
 - 考点提取 (成员2)
 - 出题与判卷 (成员1)
 - 开发
 - 错题管理 (成员2)
 - 个性化复习 (成员2)
- Agent设计 (成员1 & 成员2)
 - 出题与判卷 Agent (成员1 & 成员2)
 - 第一阶段: 核心考点提取 (成员2)
 - 第二阶段: 难度梯度控制与试卷生成 (成员1)
 - 判卷Agent (成员1)
 - 客观题: 错题深度归因机制
 - 主观题: Prometheus 评分标准
 - 复习建议生成策略Agent

4.3.1 个性化建议生成 Prompt
4.3.2 巩固效果追踪机制
三、项目亮点
3.1 核心成果对标
3.2 核心挑战与治理
3.2.1 难度梯度失控
3.2.2 “小灶”建议的针对性闭环
3.2.3 大模型幻觉问题
3.3 Prometheus 驱动的公平判卷
3.4 生产级云原生架构
四、技术难点与优化
4.1 容器化环境下的 OCR 依赖隔离问题
4.2 多模态大模型的资源调度与 OOM 优化
4.3 Redis 连接与容器健康检查
4.4 版本合并与依赖冲突管理
五、部署与运维
5.1 一键部署
5.2 数据备份
5.3 日志管理
5.4 监控与告警
附录
A. 关键文件清单
B. 端口映射
C. 环境变量

一、项目概览

1. 项目背景

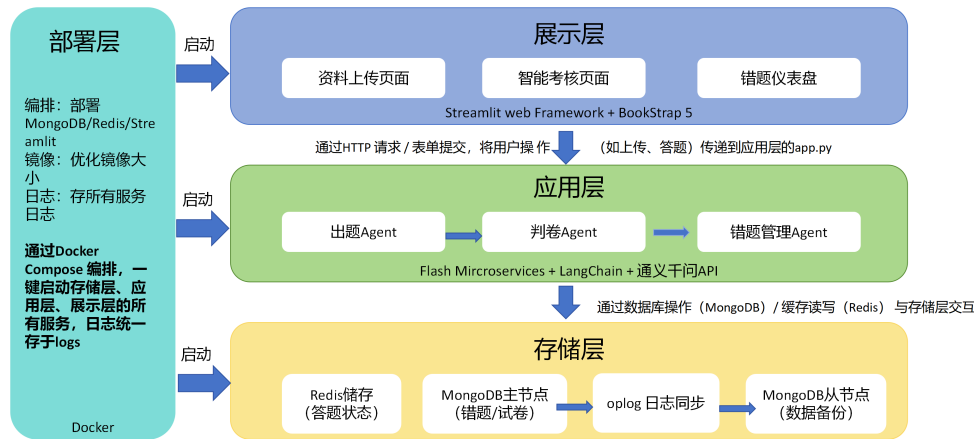
痛点场景 (Problem)	本项目解决方案 (Solution)
缺乏客观评估 学完新知识后缺乏客观的评估手段，无法发现自己的知识盲点。	动态出题与全覆盖 根据特定领域的教材、PDF 笔记或录音，自动生成覆盖核心考点的梯度试卷（选择+简答），实现“学-测”无缝衔接。
反馈深度不足 传统练习往往只给对错，缺乏详尽解析，学生难以理解错误根源。	智能判卷与深度解析 利用 LLM 及 Prometheus 提示词法进行公平判卷，不仅判断对错，更提供详尽的错误归因分析（如概念混淆、逻辑漏洞）。
复习缺乏针对性 错题记录分散，难以进行系统性的巩固与纠偏。	个性化纠偏闭环 利用 Redis/MongoDB 实现持久化的“弱点记忆”，自动生成“错题本”，并提供针对性的“小灶”复习建议，完成“补”的闭环。

本项目是一个基于云计算和人工智能技术的现代化学习辅助系统，旨在实现 "学-测-补" 一体化的学习闭环，也就是资料上传（学）→ 智能考核（测）→ 错题分析（补）的业务闭环。该系统利用大语言模型（LLM）和检索增强生成（RAG）技术，能够从用户上传的多模态学习资料（PDF文档、扫描件、录音音频）中自动提取核心考点，生成梯度化的试卷，进行智能判卷，并根据用户的答题情况提供个性化的错题分析和复习建议。

2. 架构设计

2.1 整体架构设计

整体架构图：



部署层

这是系统的“地基”，负责整个应用的运行环境与编排。

- **核心技术:** Docker & Docker Compose。
- **功能:**
 - (1) **一键启动:** 通过 `docker-compose up` 命令，统一启动右侧的展示层、应用层和存储层的所有容器。
 - (2) **编排管理:** 管理 MongoDB、Redis 和 Streamlit 服务的依赖关系（如：确保数据库先启动）。
 - (3) **日志挂载:** 将容器内的日志统一挂载到宿主机的 `logs` 目录，方便运维排查。

展示层

这是用户直接交互的界面，负责数据的输入与可视化。

- **核心技术:** Streamlit Web Framework + Bootstrap 5。
- **三大模块:**
 - (1) **资料上传页面:** 用户上传 PDF/音频，触发后端的多模态解析。
 - (2) **智能考核页面:** 展示动态生成的试卷，并实时反馈判卷结果。
 - (3) **错题仪表盘:** 可视化展示用户的能力雷达图和历史错题趋势。
- **交互方式:** 通过 HTTP 请求/表单提交，将用户操作（如点击“生成试卷”）传递给下方的应用层。

应用层

这是系统的“大脑”，包含核心的 AI 业务逻辑。

- **核心技术:** Flask Microservices + LangChain + 通义千问 API。
- **智能体协作 (Multi-Agent):**
 - (1) **出题 Agent:** 接收资料 -> 提取考点 -> 生成梯度试卷。
 - (2) **判卷 Agent:** 接收答案 -> 执行混合判卷（代码逻辑+LLM） -> 输出解析。
 - (3) **错题管理 Agent:** 收集错题 -> 分析归因 -> 生成个性化复习建议。

存储层

这是系统的“记忆”，负责数据的持久化与高可用。

- 交互: 应用层通过数据库驱动 (PyMongo/Redis-py) 读写底层数据。
- 双引擎设计:
 - Redis 缓存: 存储高频读写的答题状态 (Session State)，防止页面刷新导致答题进度丢失。
 - MongoDB 副本集: 主节点负责写入试卷数据和错题记录; 从节点通过 oplog 日志同步 实现数据备份，确保数据不丢失 (High Availability)。

2.2 云原生技术栈选型与决策 (Tech Stack Selection & Rationale)

模块	技术选型	理由
前端	Streamlit	快速构建数据驱动的交互式 Web 应用，适合 AI 原型开发。
后端	Python (Flask), LangChain	强大的 AI 生态支持，LangChain 提供了完善的 Agent 编排框架。
LLM	OpenAI API / Qwen (通义千问)	提供强大的自然语言理解和生成能力；Qwen-VL 支持视觉多模态。
数据库	MongoDB, Redis	MongoDB 适合存储非结构化 JSON 数据；Redis 提供高性能缓存。
容器化	Docker, Docker Compose	实现环境一致性，简化部署流程；多阶段构建减小镜像体积。
监控	Prometheus, Grafana, ELK	实现全链路监控、指标可视化和日志分析。

2.2.1 存储架构：高可用与数据安全

- Redis 缓存层 (State Management)
 - 架构决策: 启用 AOF (Append Only File) 持久化策略。
 - 设计理由: 考试倒计时、用户实时答题草稿 (每30秒自动保存) 属于高频写操作，必须防止容器重启导致进度丢失。
 - 配置快照 (docker-compose.yml):

```
command: redis-server --requirepass ${REDIS_PASSWORD} --appendonly yes
healthcheck:
  test: ["CMD", "redis-cli", "-a", "redis_password", "ping"]
```

- MongoDB 副本集 (Primary Business Data)
 - 架构决策: 摒弃单点数据库，采用 1主2从 (PSS架构) 的副本集模式。
 - 设计理由: 确保判卷和错题录入服务不中断；同时利用副本集机制实现数据的自动冗余备份。
 - 配置快照 (docker-compose.yml):

```
command: ["mongod", "--replSet", "rs0", "--bind_ip_all"]
environment:
  - MONGO_REPLICA_SET_NAME=rs0
```

2.2.2 应用架构：微服务解耦

- **前端框架**: 使用 **Streamlit** 快速构建交互式 Web 界面，利用 Session State 管理用户会话。
- **后端服务**: 将重计算逻辑拆分为独立的 Flask 微服务（出题、判卷、管理），避免单体应用阻塞，通过 REST API 进行通信。

2.2.3 容器化构建：多阶段构建 (Multi-stage Build)

- **架构决策**: 采用 Docker 多阶段构建策略。
- **设计理由**: 将编译环境 (Builder Stage) 与运行环境 (Runtime Stage) 分离，显著减小生产镜像体积，同时确保系统级依赖（如 OCR 需的 `poppler-utils`）的隔离性。
- **Dockerfile 引用**:

```
# Stage 1: Builder
FROM python:3.10-slim AS builder
RUN pip install --prefix=/install -r requirements.txt

# Stage 2: Runtime
FROM python:3.10-slim AS streamlit-app
COPY --from=builder /install /usr/local
# 显式注入系统级依赖
RUN apt-get install -y poppler-utils ffmpeg
```

2.3 核心业务数据流设计 (Core Data Flow Design)

2.3.1 多模态资料摄入链路

系统实现了非结构化数据到结构化知识的 ETL 流程：

1. **输入**: 用户上传 PDF 文档或 MP3 录音。
2. **处理 (IngestionService)**:
 - **音频**: 经 `ffmpeg` 转码 -> `FunASR` (Paraformer模型) 本地推理 -> 文本。
 - **文档**: 经 `PyPDFLoader` 提取 -> 若文本过少触发 `Qwen-VL` 视觉 OCR -> 文本。
3. **索引 (RAGService)**: 文本切分 (Chunking) -> Embedding 向量化 -> FAISS 向量库构建。

2.3.2 考试状态同步链路

为了防止浏览器崩溃导致数据丢失，设计了双层状态同步机制：

1. **前端缓存**: `st.session_state` 保存毫秒级交互状态。
2. **服务端快照**: 每当用户修改答案，触发 `auto_save_to_redis`，将 `user_id`、`exam_content`、`user_answers` 序列化存入 Redis。
3. **持久化归档**: 提交试卷后，清洗后的评分结果写入 MongoDB `exam_records` 集合。

2.4 高并发与稳定性优化策略 (Optimization Strategy)

针对大模型应用特有的资源消耗大、延迟高的问题，我们在工程层面实施了多级优化：

2.4.1 资源调度与隔离 (Resource Isolation)

- Docker 资源限制:** 通过 `deploy.resources` 严格限制各容器的 CPU 和内存配额，防止因大模型推理（如 FunASR 占用 2GB+ 内存）导致宿主机 OOM (Out of Memory) 崩溃。
- 配置快照** (`docker-compose.yml`):

```
deploy:
  resources:
    limits:
      cpus: "1.0"
      memory: "1G" # 硬限制，防止内存泄漏
    reservations:
      cpus: "0.5"
      memory: "512M" # 资源预留，确保QoS
```

2.4.2 性能优化 (Performance Tuning)

- 懒加载单例模式 (Lazy Loading):**
针对 FunASR 和 Vision LLM 等重型模型，不在应用启动时加载，而是推迟到首次请求时初始化。这解决了容器启动超时 (Timeout) 问题。
 - 代码引用:** `IngestionService` 中 `if self.asr_model is None: load_model()`。
- 向量化批处理 (Batch Processing):**
在构建 RAG 索引时，避免单条调用 Embedding API，而是采用分批处理 (Batch Size = 25/64)，显著减少网络 I/O 开销并防止 API Rate Limit 限制。
 - 代码引用:** `rag_service.py` 中的 `embeddings.embed_documents(batch)`。
- 连接池复用:**
MongoDB 和 Redis 客户端均配置了连接池，避免因频繁握手导致的性能损耗。

二、核心技术实现

1 团队分工总览 (组内均分)

角色	成员1——何峻伟 (算法/Agent组, 35%)	成员2——黄煜 (算法/Agent组, 35%)	成员3——安雪娇 (架构/工程组, 30%)
核心职责	基础框架搭建、负责pdf读取、LLM出题与判卷逻辑	负责录音读取、考点提取、错题管理与个性化复习	负责存储架构+前端交互+部署优化

角色	成员1——何峻伟 (算法/Agent组, 35%)	成员2——黄煜 (算法/Agent组, 35%)	成员3——安雪娇 (架构/工程组, 30%)
具体工作	1. 基础框架搭建; 2. pdf的读取 (包括扫描件); 3. 设计考点提取+出题Prompt (确保覆盖核心知识点); 4. 开发 Prometheus判卷提示词; 5. 主观题解析生成逻辑	1. 录音读取; 2. 考点读取; 3. 难度分级规则库设计; 4. 错题数据结构 (关联考点/章节); 5. 个性化复习建议生成Prompt	1. 搭建Redis+MongoDB存储集群; 2. 开发Web答题界面 (支持上传资料/答题/查看错题); 3. 编写Docker配置+环境部署文档

2 架构实现 (成员3)

2.1 存储架构高可用升级

- **MongoDB 副本集重构**
 - 将成员1初始的单点 MongoDB 升级为 **1主2从 (PSS架构)** 的副本集 (`replicaSet=rs0`), 实现了数据的自动故障转移与读写分离基础。
- **Redis 持久化增强**
 - 开启 Redis **AOF (Append Only File)** 持久化策略, 确保在容器重启时, 用户的实时答题状态 (Session State) 不丢失。
- **连接池优化**
 - 并在代码中配置了 `maxPoolSize=200` 和自动重连机制, 解决了高并发下的连接耗尽问题。

2.2 前端交互与可视化深度优化

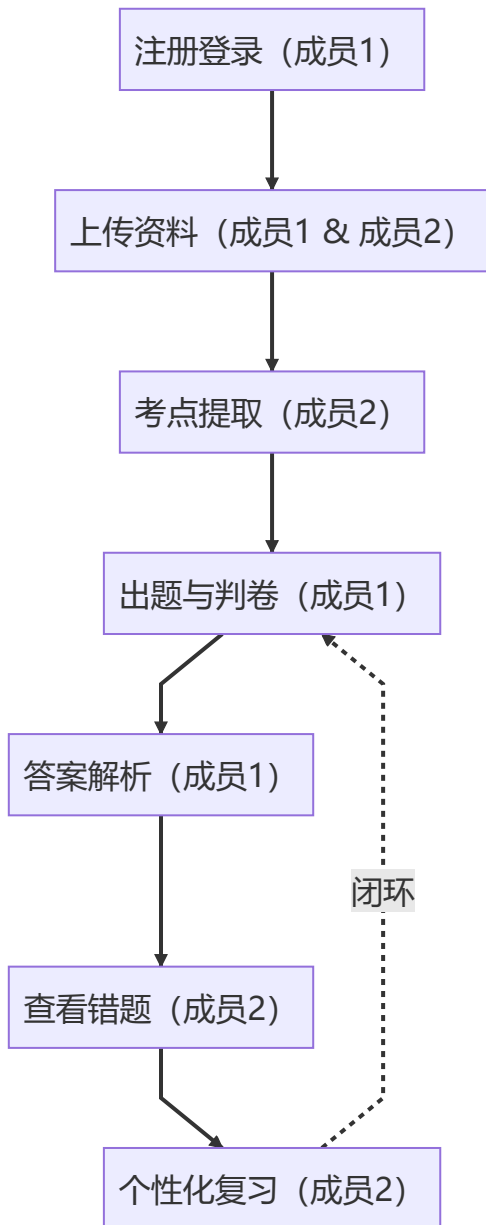
- **仪表盘 (Dashboard) 开发**
 - 引入 **Plotly** 交互式图表库, 实现了错题难度分布 (饼图/柱状图) 和学习进步曲线的动态渲染, 替代了静态文本展示。
- **用户体验 (UX) 升级**
 - 实现了“答题进度条”、“实时草稿保存提示”以及“玻璃态 UI”样式, 大幅提升了系统的交互质感。
 - 开发了“错题导出”功能, 解决了 CSV 文件在 Excel 中打开中文乱码的编码问题 (`utf-8-sig`)。

2.3 容器化运维与资源治理

- **Dockerfile 多阶段构建优化**
 - 重构 Dockerfile, 分离 `builder` (编译层) 和 `runtime` (运行层), 并将 `ffmpeg`、`poppler-utils` 等系统依赖注入到底层镜像, 解决了 OCR 功能在容器内报错 `PDFInfoNotInstalledError` 的关键问题。
- **资源配额管理**
 - 在 `docker-compose.yml` 中为 `FunASR` 等重型服务配置了 `deploy.resources`, 限制内存上限为 1GB, 防止因模型推理导致的宿主机 OOM (内存溢出)。
 - 配置了完善的 `healthcheck` (健康检查) 探针, 确保服务按依赖顺序启动 (Redis -> Mongo -> App)。

3 算法实现 (成员1 & 成员2)

整体算法流程图:



3.1 用户注册与登录 (成员1)

- **逻辑流程:**

1. **注册:** 用户输入用户名、密码、邮箱等信息 -> 后端校验唯一性 -> 使用 `bcrypt` 对密码进行加盐哈希 -> 存入 MongoDB `users` 集合。
2. **登录:** 用户输入凭证 -> 后端从数据库检索哈希密码 -> 使用 `bcrypt.checkpw` 验证 -> 验证通过后将用户信息写入 Streamlit `session_state` 维持会话状态。

3.2 上传资料 (成员1 & 成员2)

支持pdf、音频文件上传

上传流程

1. 用户在前端页面选择文件（支持批量上传）。
2. 前端将文件发送至后端 API。
3. 后端接收文件后，根据文件类型（PDF/音频）调用不同的处理函数。
 - **PDF**: 使用 `PyPDF2` 提取文本内容，调用 `Qwen-VL` 进行 OCR 识别（若包含图片）。
 - **音频**: 使用 `FunASR` 进行语音识别，将文本内容保存至数据库。
4. 处理完成后，返回上传成功提示，用户可在前端查看已上传的文件。

单个pdf文件上传（成员1）

- **处理逻辑**: 使用 `PyPDFLoader` (LangChain) 直接提取文本流。
- **质量检测**: 系统自动检测提取文本的长度，若有效字符数 < 50 ，则智能判定为“扫描件”，建议用户开启 OCR 模式。
- **入库**: 提取的纯文本被清洗后存入 MongoDB `knowledge_base` 集合。

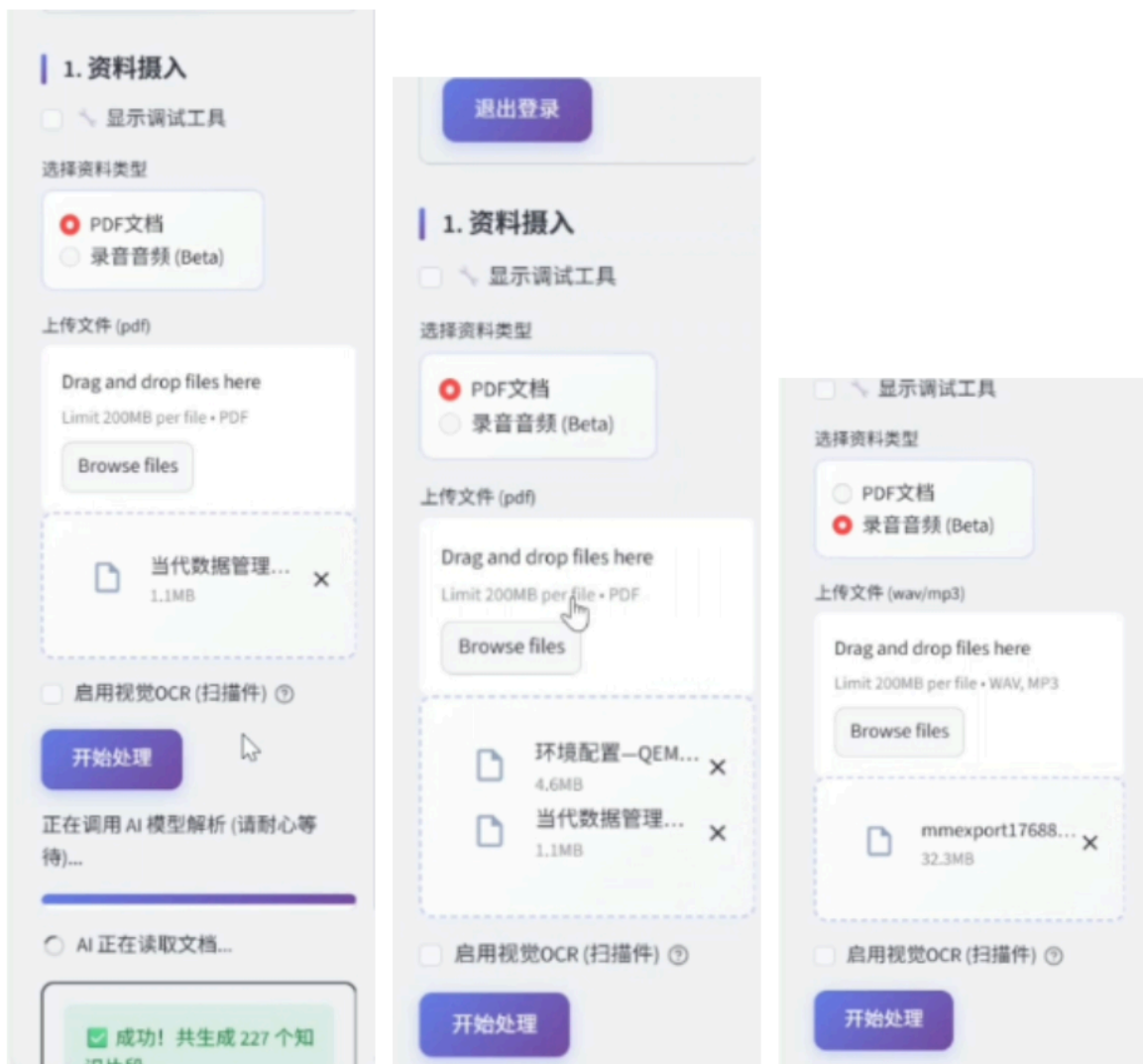
多个pdf上传（成员2）

- **批处理**: 前端 `st.file_uploader` 开启 `accept_multiple_files=True`，支持一次性上传多份教案或课件。
- **迭代处理**: 后端遍历文件列表，复用单个 PDF 的处理逻辑，并实时显示处理进度条，确保大批量上传时的用户体验。

录音上传（成员2）

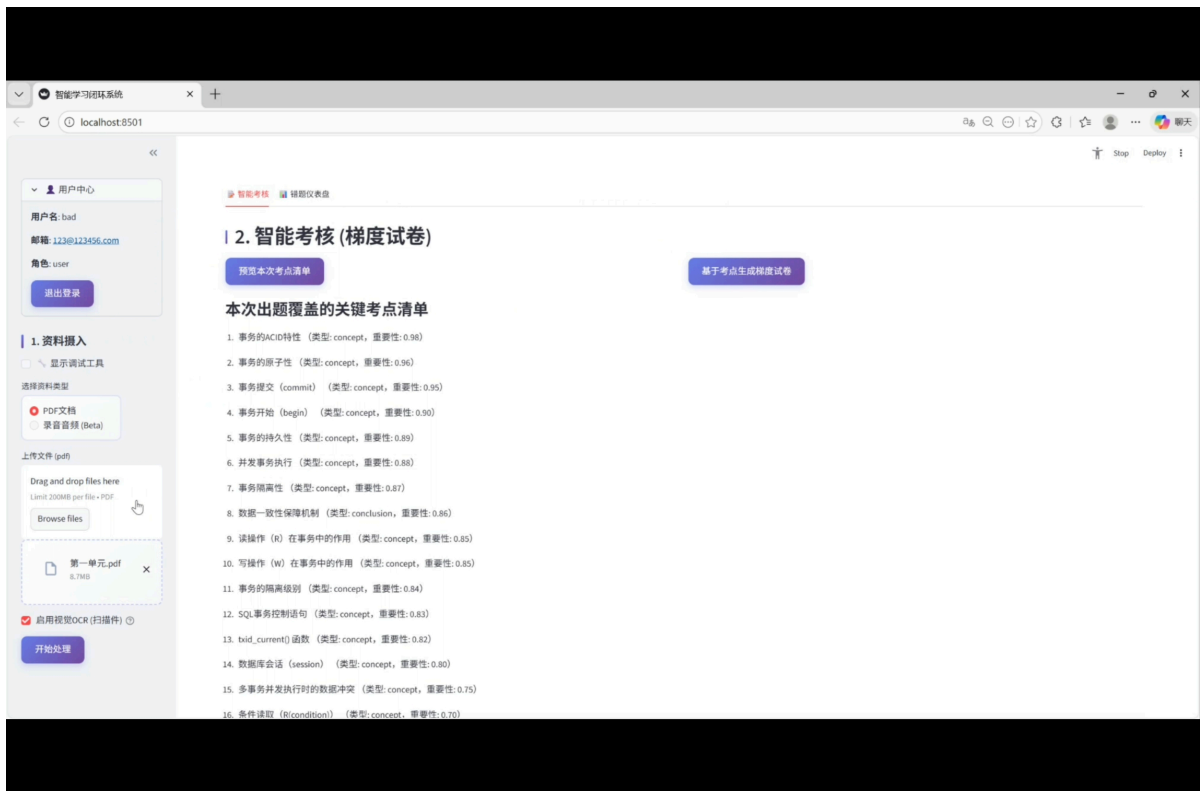
- **技术栈**: `FunASR` (Paraformer 模型) + `ffmpeg`。
- **流程**:
 1. **转码**: 上传的音频 (如 MP3) 首先通过 `pydub` 调用系统级 `ffmpeg` 转码为 **16k Hz 单声道 WAV**，以适配 ASR 模型输入要求。
 2. **推理**: 使用阿里达摩院的 **FunASR** 引擎进行本地推理，生成带标点符号的会议纪要/课堂实录。
 3. **优化**: 模型采用 **Lazy Loading** (懒加载) 策略，仅在首次使用时加载进内存，降低闲置资源占用。

下面是有关以上三种类型的上传截图：



扫描件上传 (成员1)

- **技术栈:** pdf2image + Qwen-VL (视觉大模型)。
- **视觉模型接入**
 - 在 IngestionService 中集成了 Qwen-VL (通义千问视觉模型), 实现了对扫描件 PDF 和纯图片文档的文字提取能力。
- **智能回退机制 (Fallback Logic)**
 - 开发了智能判断逻辑: 系统优先使用 PyPDFLoader 解析文档, 当检测到有效字符数小于 50 (判定为扫描件) 时, 自动切换至视觉 OCR 模式, 平衡了识别速度与精度。
- **图像预处理流水线**
 - 引入 pdf2image 库将 PDF 页面转换为图片流, 并结合 io.BytesIO 实现内存级 Base64 编码传输, 避免了大量临时文件的磁盘 I/O 开销。



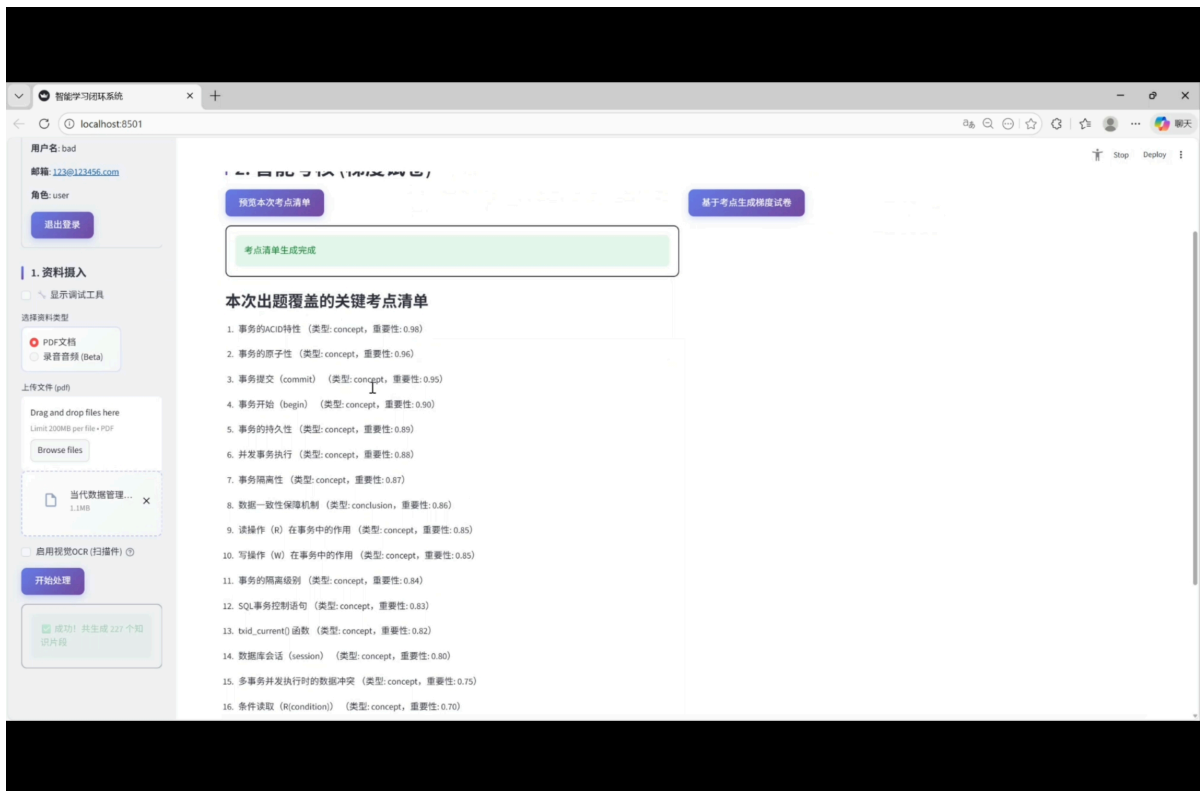
3.3 考点提取（成员2）

- **核心逻辑:**

- 采用 **Two-Stage CoT (思维链)** 策略，首先从非结构化文本中提取结构化考点，再基于考点生成试题，避免了 LLM 直接生成试卷时的“幻觉”和“跑题”问题。

- **Prompt 设计:**

- 让 Agent 扮演“教学设计专家”，从长文本中提取 Top-N 核心考点。
- **权重机制:** 引入 `importance` (0-1) 权重，确保生成的试卷优先考察高价值知识点（如核心定义、关键公式）。
- **结构化输出:** 强制要求输出 JSON 格式 `[{"keyword": "...", "importance": 0.9, "type": "concept"}]`，便于后续程序处理。

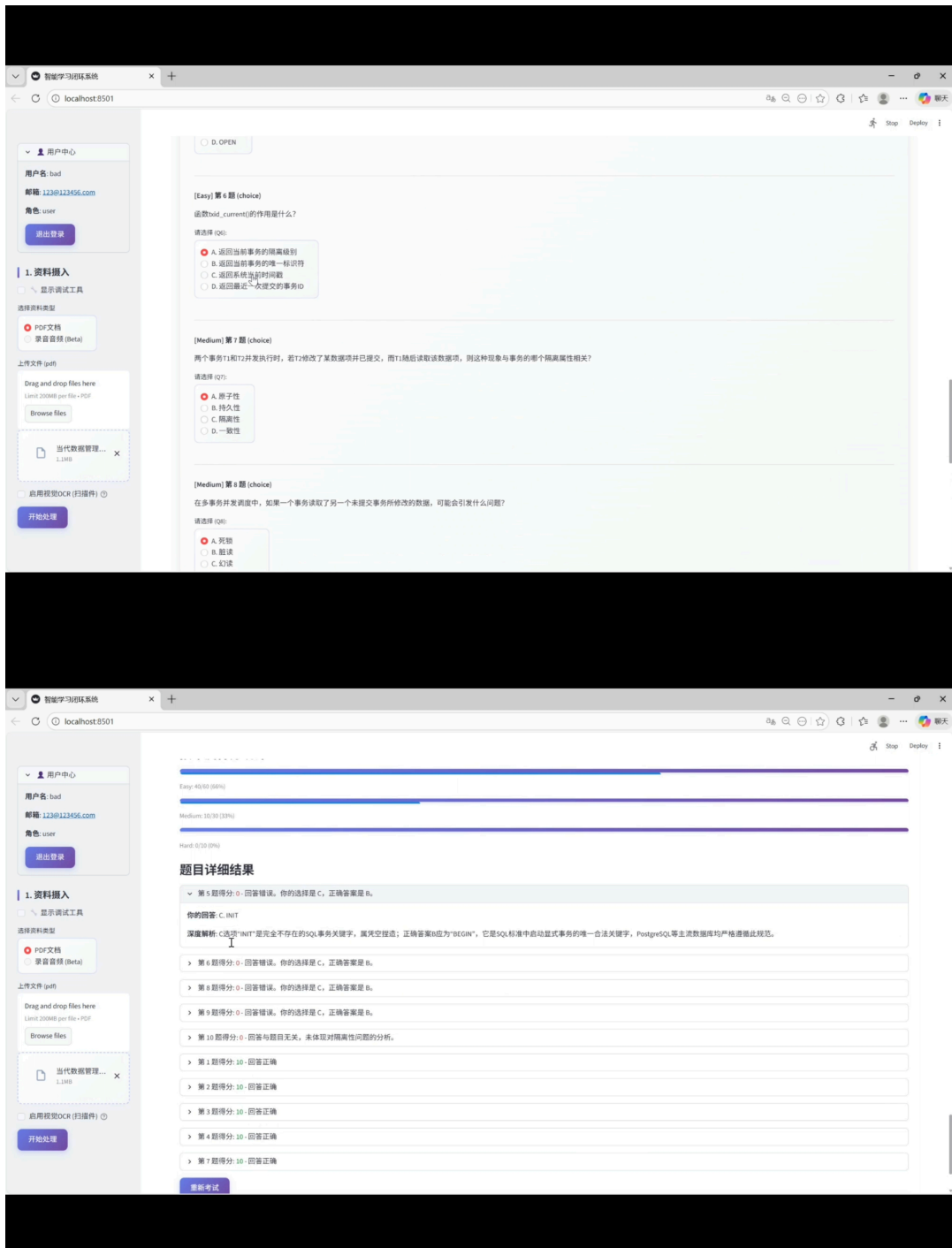


3.4 出题与判卷（成员1）

开发

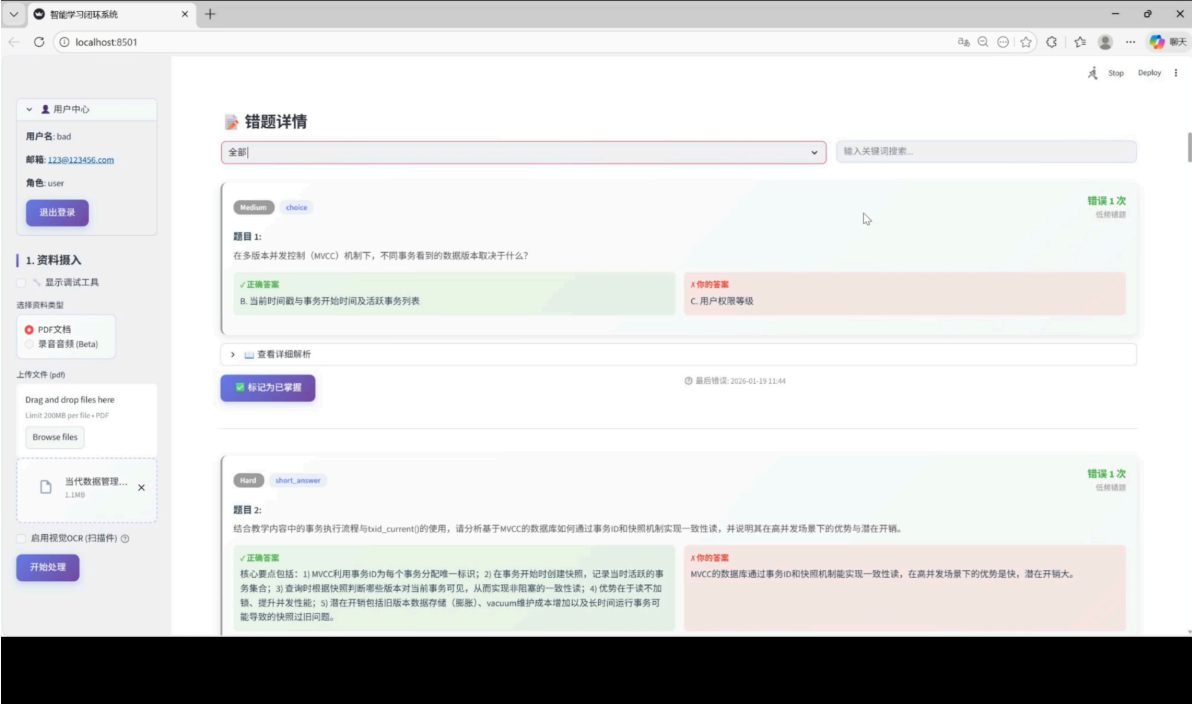
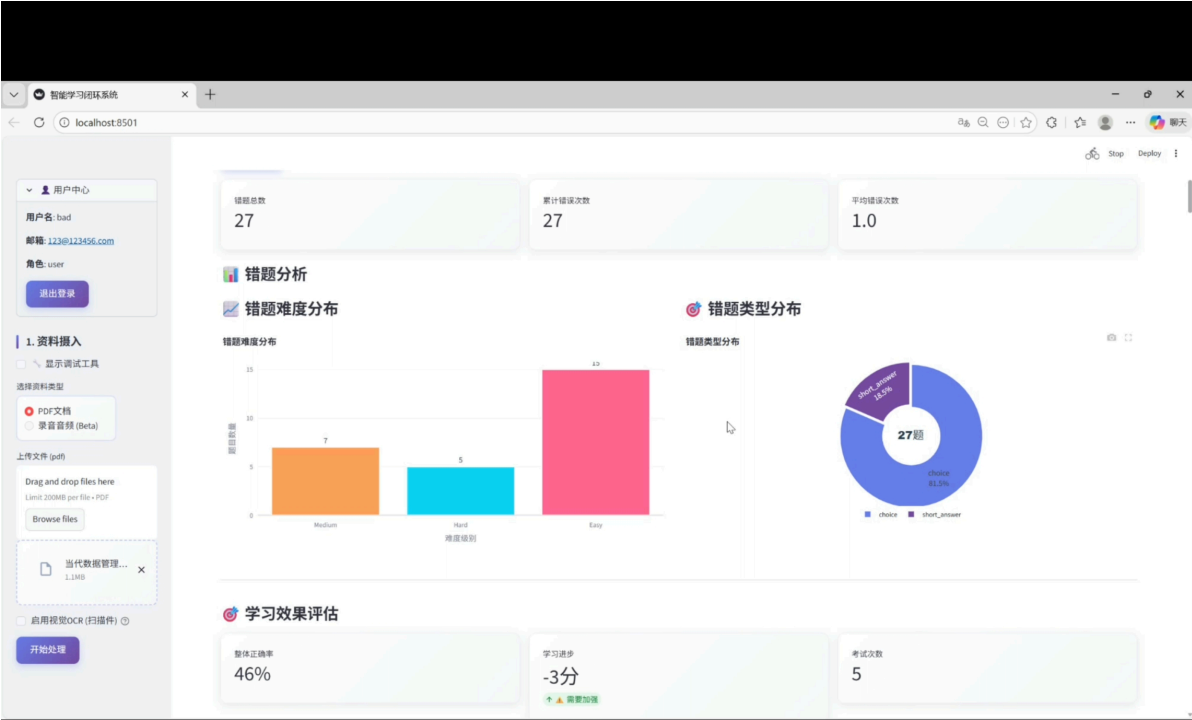
- **出题 Agent (QuizGenerator) 开发**
 - 实现了 "Keypoint-Driven" (考点驱动) 的双阶段出题策略：先提取 keyword 和 importance 权重，再生成试卷，有效解决了大模型出题跑题的问题。
 - 硬编码了 DIFFICULTY_RULES (难度规则字典)，强制实现了 6:3:1 (基础:进阶:挑战) 的难度分布，确保试卷结构的合理性。
 - 设计了严格的 JSON 输出 Prompt，解决了大模型生成选项格式混乱（如缺失 "A." 标签）导致前端渲染失败的工程难题。
- **判卷 Agent (QuizGrader) 开发**
 - 设计了 "Hybrid Grading" (混合判卷) 策略：客观题优先使用 Python 字符串硬匹配（节省 Token），仅在错题时调用 LLM 进行深度归因分析；主观题强制调用 LLM。

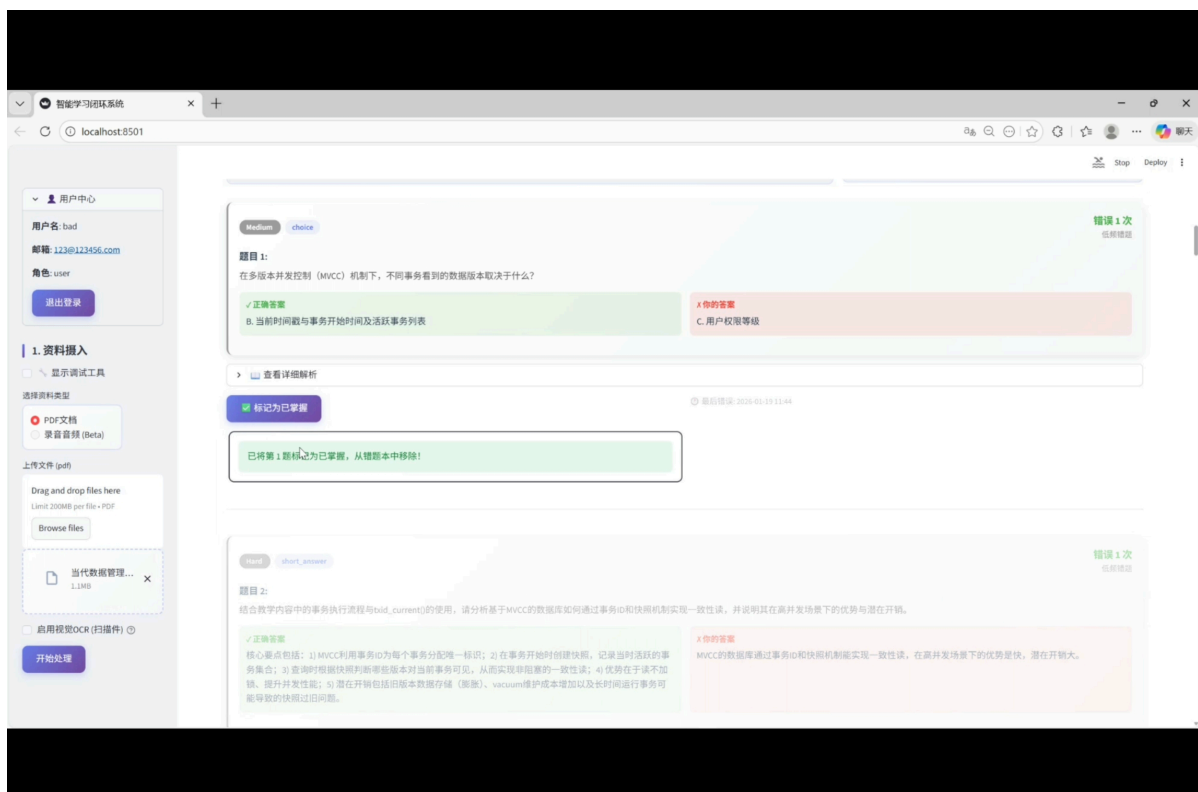
- 引入 **Prometheus 评分标准**，通过 Prompt 约束模型从“知识覆盖”、“逻辑连贯”、“准确性”三个维度输



3.5 错题管理（成员2）

- 多维度数据可视化 (DashboardView):
 - 学习进度概览: 展示总考试次数、错题总数、平均分数及最近一次得分，直观反映学习状态。
 - 考试历史趋势: 使用 Plotly 绘制分数折线图，追踪用户在时间维度上的成绩变化。
 - 错题分布分析: 提供“难度分布”与“题型分布”饼图，帮助用户识别薄弱环节（如“挑战题错误率高”或“概念混淆”）。
- 错题详情与交互:
 - 错题聚合: 将同一知识点的多次错误聚合展示，显示累计错误次数与最近错误时间。
 - 标记掌握: 提供手动“标记为已掌握”功能，允许用户移除已攻克的错题，保持错题本的精简与高效。





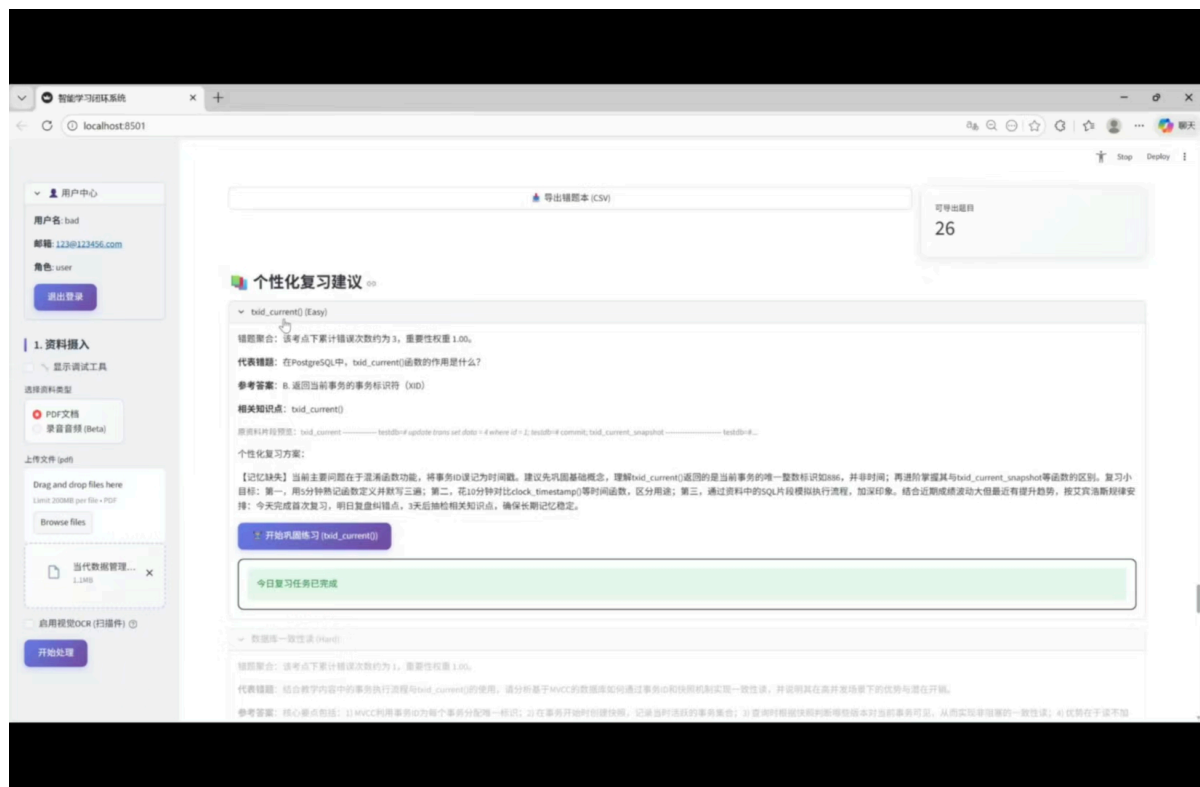
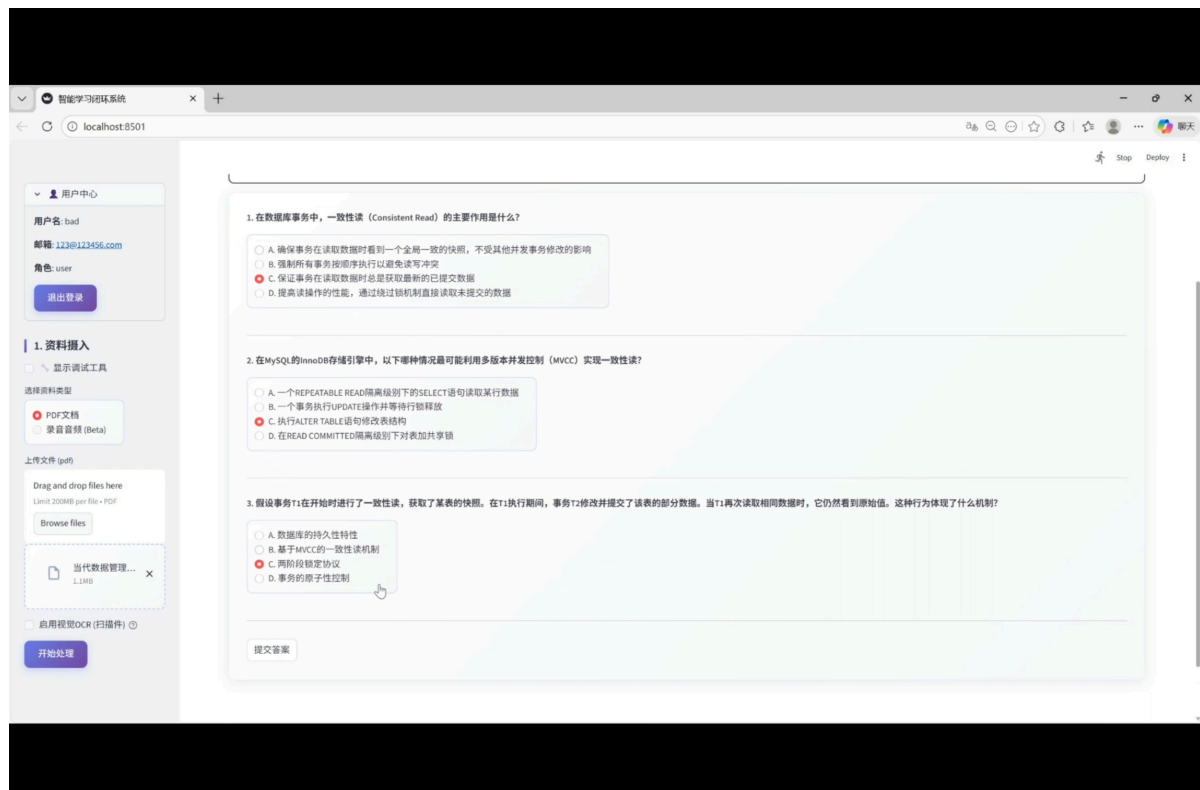
- 第一张图为错题仪表盘中的难度分布可视化：直观地告诉用户，自己的错误主要集中在哪个难度层级（简单/中等/困难）。如果“简单题”错误率高，说明基础不牢；如果“困难题”错误率高，说明综合应用能力有待提升。这有助于用户识别自己的薄弱环节。
- 第二张图为错题详情（展示单道错题的详细解析卡片）：该界面不仅显示题目、用户错误选项和正确答案，更重要的是展示了 Agent 生成的深度归因分析（例如：“你混淆了概念A和B”）。这是“智能判卷”的具体体现，帮助用户从根本上理解错误原因。
- 第三张图为错题本的 **交互功能**：用户可以点击“标记为已掌握”按钮，将错题从错题本中移除。这是一个手动移除错题的操作演示。当用户认为自己已经彻底学会了某道错题时，可以点击“标记为已掌握”，将其从待复习列表中移除。这实现了错题本的 **动态瘦身**，确保用户只关注尚未解决的问题，提高复习效率。

3.6 个性化复习（成员2）

- **Review Agent (复习建议智能体) 开发**
 - 构建了基于 **用户画像 + 错题聚类** 的个性化推荐引擎，而非简单的错题罗列。
 - **Prompt 设计**:
 - 注入了丰富的上下文信息：`behavior_summary` (学习状态，如“近期成绩波动较大”)、`weak_point` (薄弱考点)、`error_times` (错误次数) 以及 `relevant_content` (RAG 检索到的教材原文)。
 - 要求 Agent 生成 **"诊断-行动-目标"** 三段式建议，例如：“你混淆了 IaaS 和 PaaS 的管理边界（诊断），建议重读教材第 3 章图表（行动），争取下次正确率达 80%（目标）”，并且学习目标分为**长期**和**短期**两种，满足用户需求。
- **巩固练习与任务打卡**:
 - **针对性巩固 (Consolidation Quiz)**: 用户可点击“开始巩固练习”按钮，系统即时生成针对特定薄弱点的 3 道新题，进行强化训练。
 - **学习任务打卡**: 系统为每个薄弱点生成复习任务，用户完成复习后可打卡，系统记录打卡状态并更新学习路径。
 - **闭环验证**:

- 引入 "Impact Tracking" (效果追踪) 机制，通过对比“小灶前”与“小灶后”的正确率变化，验证个性化复习的实际效果，并动态调整推荐策略。

以下两张图分别为小灶（巩固练习）与复习打卡（学习任务打卡）的可视化界面：



4. Agent设计 (成员1 & 成员2)

4.1 出题与判卷 Agent (成员1 & 成员2)

第一阶段：核心考点提取 (成员2)

直接让模型生成试卷往往会导致考点分散或偏离教材。我们首先让 Agent 扮演“教学设计专家”，从长文本中提取核心知识点及其重要性权重。

Prompt 设计 (源码引用):

```
"""
你是一名课程教学设计专家。请从以下[教学内容]中提取核心考点（包括重要概念、关键公式、核心结论），
并按照重要性从高到低排序。

[教学内容]
{context}

提取要求：
1. 覆盖主要章节的关键知识点，避免遗漏重要概念。
2. 每个考点给出 0 到 1 之间的 importance 权重，越重要越接近 1。
3. 标注考点类型，限定为 "concept"、"formula"、"conclusion" 或 "other" 之一。
4. 控制考点数量不超过 {max_points} 个。

输出格式：
直接返回 JSON 数组，不要添加任何说明文字...
"""
```

设计合理性分析:

- 知识锚点:** 提前提取 `keyword` 和 `importance`，为后续出题提供了“知识锚点”，防止模型在长文本中迷失重点。
- 权重机制:** 引入 `importance` 权重，确保生成的试卷优先考察高价值知识点，避免考察边缘细枝末节。

第二阶段：难度梯度控制与试卷生成 (成员1)

基于提取的考点，我们设计了极其严格的 Prompt 来控制试卷结构，确保智能体所给出的题目具有一定的“难度梯度”。

Prompt 设计 (源码引用):

```
"""
你是一个专业的出题专家。请基于以下[教学内容]，生成一套包含 **10道题目** 的完整试卷。

[教学内容]: {context}
[提取的核心考点列表]: {keypoints}

[出题结构与要求]:
请严格按照以下顺序和难度出题...使整体难度分布满足“基础题约60%、进阶题约30%、挑战题约10%”：

1. 题目 1-6 (共6题):
  - 类型: 单选题 (Choice)
  - 难度: 简单 (Easy)
  - 目标: 考察基本概念、定义和术语记忆。

```

- 要求：必须包含 A/B/C/D 四个选项，且正确答案分布要均衡...

2. 题目 7-9 (共3题):

- 类型：单选题 (Choice)
- 难度：中等 (Medium)
- 目标：考察概念对比、场景应用或简单计算。

3. 题目 10 (共1题):

- 类型：简答题 (Short Answer)
- 难度：困难 (Hard)
- 目标：考察综合理解、架构设计或优缺点深度分析。

[输出格式]:

请严格返回一个包含10个对象的JSON数组...

"""

设计合理性分析:

- **显式梯度控制:** Prompt 中硬性规定了 6:3:1 的难度分布和对应的考察目标 (记忆 vs 应用 vs 分析)，直接响应了命题二中“出题的难度梯度是否合理”的考核指标。
- **反位置偏见:** 显式要求“正确答案分布要均衡”，防止大模型倾向于生成 'A' 或 'B' 选项，提高了试卷的专业度。
- **工程鲁棒性:** 严格限制 JSON 格式和 Option 格式 ("A.", "B."...)，确保前端 Streamlit 渲染时不会崩溃。

4.2 判卷Agent (成员1)

“智能判卷”的要求为不仅给出对错，更要给出详尽解析，同时兼顾工程落地的成本与响应速度，我们设计了一套“**确定性逻辑 + 生成式AI**”的混合判卷策略 (Hybrid Grading Strategy)。

客观题：错题深度归因机制

对于选择题，系统优先使用 Python 字符串比对进行硬逻辑判断。仅当用户**答错**时，才会触发 Agent 调用。此时 Agent 的角色不是简单的“裁判”，而是“辅导老师”，专注于分析用户的**错误原因**（是概念混淆、计算错误还是逻辑陷阱）。

Prompt 设计 (源码引用):

"""

你是一个专业的辅导老师。学生在一道选择题时选错了。

[题目信息]

- 题目: {question}
- 正确答案: {correct_answer}
- 学生错误选项: {user_answer}
- 相关知识点: {context}

[任务]

1. 明确指出学生的选项({user_answer})错在哪里 (例如: 概念混淆、计算错误、逻辑陷阱等)。
2. 简要解释正确答案({correct_answer})的依据。
3. 解析要一针见血，控制在100字以内。

[输出]

直接输出解析文本，不要包含任何格式标记。

```
"""
```

设计合理性分析:

- 角色设定 (Persona):** 设定为“辅导老师”而非“阅卷机”，引导模型输出更具教育意义的反馈，而非冷冰冰的“错误”。
- 上下文注入 (Context Injection):** 注入 `{context}` (知识点背景) 和 `{user_answer}` (错误选项)，强制模型进行对比分析，解决“只知其然不知其所以然”的痛点。
- 成本优化:** 答对时不调用 LLM，仅在答错时调用，大幅降低 Token 消耗和系统延迟。

主观题：Prometheus 评分标准

针对简答题，为了实现命题二要求的“公平性”，我们采用了 **Prometheus 提示词法**。通过显式定义评分维度和权重，强制 LLM 输出结构化的 JSON 评分报告，避免大模型打分的随意性。

Prompt 设计 (源码引用):

```
"""
### Role
你是一个公正的考官。请使用 Prometheus 评分标准评估学生的简答题。

### Input
- 题目: {question}
- 参考答案: {correct_answer}
- 学生回答: {user_answer}
- 知识背景: {context}

### Scoring Criteria (Total 10分)
1. 知识点覆盖 (0-5分): 关键词命中情况
2. 逻辑连贯性 (0-3分): 表达清晰度
3. 准确性 (0-2分): 是否有事实错误

### Output Format (JSON Only)
请严格返回合法的JSON格式，不要包含Markdown标记:
{{
    "score": <0-10>,
    "feedback": "简短评语",
    "analysis": "详细解析: 失分点在哪里..."
}}
```

设计合理性分析:

- 多维度量化 (Multidimensional Metric):** 将模糊的“好/坏”拆解为“知识覆盖”、“逻辑”、“准确性”三个维度，符合教学评估标准。
- 结构化输出 (Structured Output):** 强制 JSON 输出，使得后端可以直接解析分数 (`score`) 用于统计，解析内容 (`analysis`) 用于前端展示，实现了工程上的闭环。
- 公平性保障:** 固定的评分标准 (`Scoring Criteria`) 减少了模型幻觉导致的评分波动。

4.3 复习建议生成策略Agent

“针对错题的‘小灶’建议是否有针对性”的考核指标，系统不仅仅是简单地罗列错题，而是构建了一个**基于用户行为数据和错题聚类的个性化推荐引擎**。该模块通过分析用户的历史答题数据、错题分布和学习行为，动态生成复习计划。

#####多维度错题聚合与分析

系统首先对用户的错题数据进行深度清洗和聚合，而非孤立地看待每一道错题。通过 `get_weak_points` 接口，系统识别出用户的高频薄弱点。

- **数据输入源:**
 1. **错题元数据:** 包含错误次数 (`error_times`)、题目难度 (`difficulty`)、考点重要性 (`importance`)。
 2. **用户行为画像:** 包含总考试次数、平均分、最近一次得分以及**成绩波动情况**（如“最近得分波动较大”或“保持稳定”）。
 3. **知识关联:** 错题对应的章节 (`chapter`)、小节 (`section`) 以及知识库中的相关原文片段 (`relevant_content`)。

4.3.1 个性化建议生成 Prompt

我们设计了一个包含丰富上下文的 Prompt，让 LLM 扮演“学习顾问”，根据用户的具体表现生成具有针对性的复习方案，而非通用的“多看书”。

Prompt 设计:

```
"""
你是一位专业的学习顾问。请根据以下用户的学习数据，为考点“{keyword}”生成一份个性化的短时复习计划。

[用户画像]
- 学习状态: {behavior_summary}（例如：近期成绩波动较大）
- 薄弱考点: {keyword}（难度: {difficulty}, 重要性权重: {importance}）
- 累计错误: {error_times} 次
- 最后一次错误时间: {last_error_time}

[代表性错题]
- 题目: {sample_question}
- 用户错误答案: {sample_user_answer}
- 标准答案: {sample_standard_answer}
- 错误原因解析: {sample_analysis}

[知识库原文片段]
{relevant_content}

[生成要求]
1. **诊断**: 一针见血地指出用户在该考点上的主要认知误区（结合代表性错题）。
2. **行动**: 制定 3 步具体的复习行动（例如：重读教材第X章、对比记忆X概念、手动推导X公式）。
3. **目标**: 设定一个可量化的短期目标（例如：在下次巩固练习中正确率达到80%）。
4. **语气**: 鼓励且务实，避免空泛的说教。

[输出格式]
请直接输出建议内容，使用 Markdown 格式，包含**诊断**、**行动计划**、**预期目标**三个小标题。
"""
```

设计合理性分析:

- 情境感知 (Context Aware):** Prompt 中注入了 `behavior_summary` (如“近期波动较大”), 使得生成的建议能感知用户的整体状态 (是需要稳扎稳打还是急需突破), 极大地提升了“针对性”。
- 证据驱动 (Evidence Based):** 引用具体的 `sample_question` 和 `sample_user_answer`, 让 LLM 的诊断基于具体事实 (“你在区分 IaaS 和 PaaS 时混淆了管理边界”), 而非臆断。
- 闭环行动 (Actionable):** 强制要求输出“行动计划”和“预期目标”, 引导用户从“看建议”转向“做练习”, 配合系统中的 “📌 开始巩固练习” 按钮, 真正实现了从评估到提升的闭环。

4.3.2 巩固效果追踪机制

为了验证复习建议的有效性, 系统引入了**前测-后测对比机制** (Code Reference: `render_consolidation_history`) 。

- 小灶前正确率:** 基于历史考试中该考点的平均正确率。
- 小灶后正确率:** 用户在完成针对性“巩固练习”后的正确率。
- 可视化反馈:** 系统使用 Plotly 绘制对比柱状图, 直观展示复习前后的正确率变化 (如“提升了 30%! ”), 并给出动态评语 (“显著进步”、“保持平稳”或“状态起伏”), 以此正向激励用户持续学习。

三、项目亮点

3.1 核心成果对标

命题要求	本项目实现成果	技术支撑 (Code Reference)
输入要求 (教材/笔记/录音)	多模态全覆盖: 支持 PDF 文档 (含扫描件 OCR) 与 MP3 音频录音的自动摄入与解析。	<code>IngestionService</code> 集成了 <code>Qwen-VL</code> 视觉模型与 <code>FunASR</code> 语音模型
动态出题 (覆盖核心考点)	双阶段生成策略: 先提取 Knowledge Graph 锚点, 再生成题目, 确保不偏题、不产生幻觉。	<code>QuizGenerator</code> 实现了 <code>extract_keypoints -> generate_exam</code> 的 CoT 链路
智能判卷 (详尽解析)	混合判卷引擎: 结合确定性逻辑与生成式 AI, 提供针对性的错误归因分析。	<code>QuizGrader</code> 采用 Python 硬逻辑校验 + LLM 错题深度辨析
云原生技术 (持久化记忆)	高可用存储集群: 构建了 MongoDB 副本集与 Redis AOF 持久化, 确保学习数据不丢失。	<code>docker-compose.yml</code> 配置了 <code>mongo-replset</code> 和 <code>redis-appendonly</code>

3.2 核心挑战与治理

针对 AI 应用常见的幻觉、随机性等问题, 本项目实施了严格的治理策略, 确保系统满足考核指标:

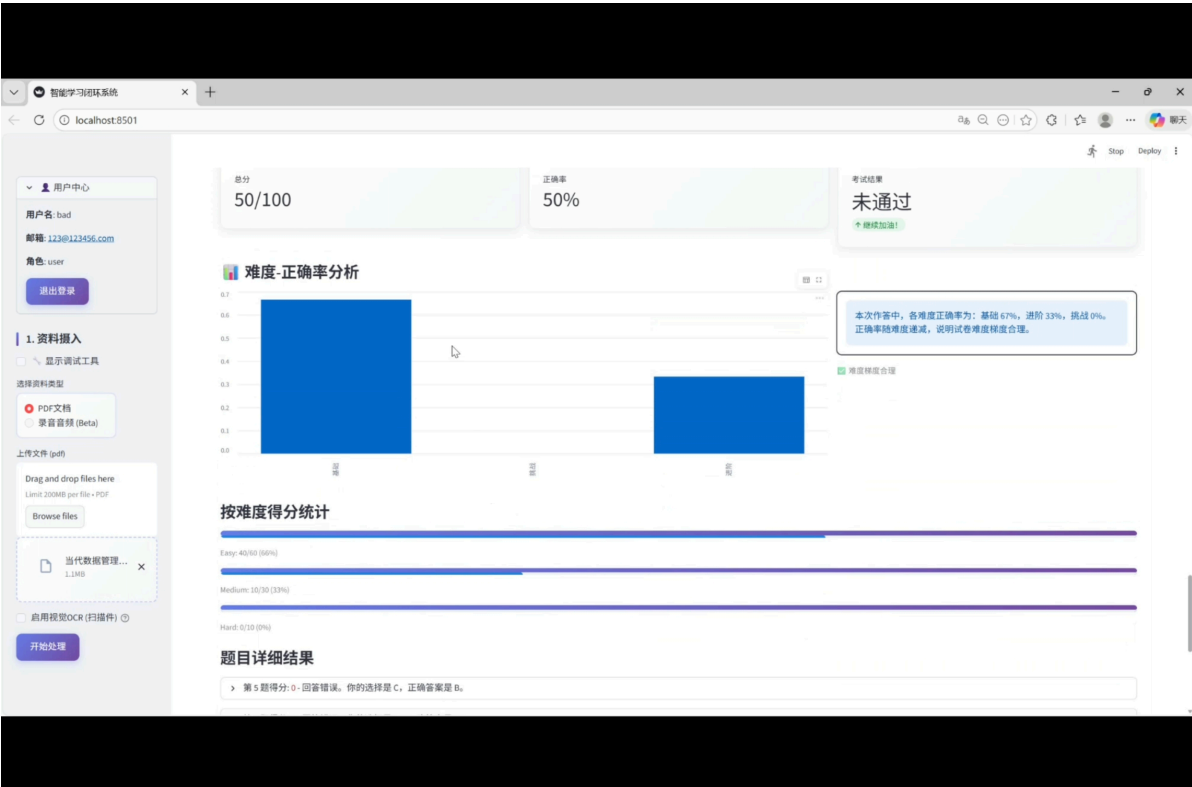
挑战 (Challenge)	应对策略 (Strategy)
难度梯度失控 (LLM 倾向于生成简单题)	结构化约束 + 后验分析 1. 强制规则: 在 Prompt 中硬性植入 <code>DIFFICULTY_RULES</code> (6:3:1 比例)。 2. 知识锚点: 引入权重机制, 确保高难度题目对应高权重考点, 防止“简单题考难点”或“难题考偏点”。

挑战 (Challenge)	应对策略 (Strategy)
建议缺乏针对性 (生成“请多复习”等通用废话)	归因驱动的反馈闭环 1. 精准归因 : 判卷时强制分类错误类型 (如: 概念混淆/计算错误/记忆偏差)。 2. 上下文增强 : 将错题对应的 原文片段 重新注入 Prompt, 要求生成具体到“章节-概念”的复习路径。
大模型幻觉 (一本正经胡说八道)	RAG 溯源 + 混合判卷 1. 知识落地 : 强制 LLM 仅基于检索到的 Context 作答, 禁止利用训练数据中的外部知识。 2. 确定性兜底 : 客观题优先使用代码逻辑判分, 完全规避 了判卷阶段的幻觉风险。

3.2.1 难度梯度失控

对于出题的难度梯度是否合理的判断, 我们摒弃了随机生成, 实施了严格的**结构化控制**:

- 6:3:1 黄金比例**: 代码中硬编码了 `DIFFICULTY_RULES`, 强制生成 60% 基础题 (考察记忆)、30% 进阶题 (考察应用)、10% 挑战题 (考察分析)。
- 后验分析机制**: 判卷服务包含 `analyze_difficulty_gradient` 模块, 自动计算各难度层级的正确率分布 (如: 基础题 90% vs 挑战题 40%), 实时验证梯度的合理性, 防止出现“倒挂”现象。

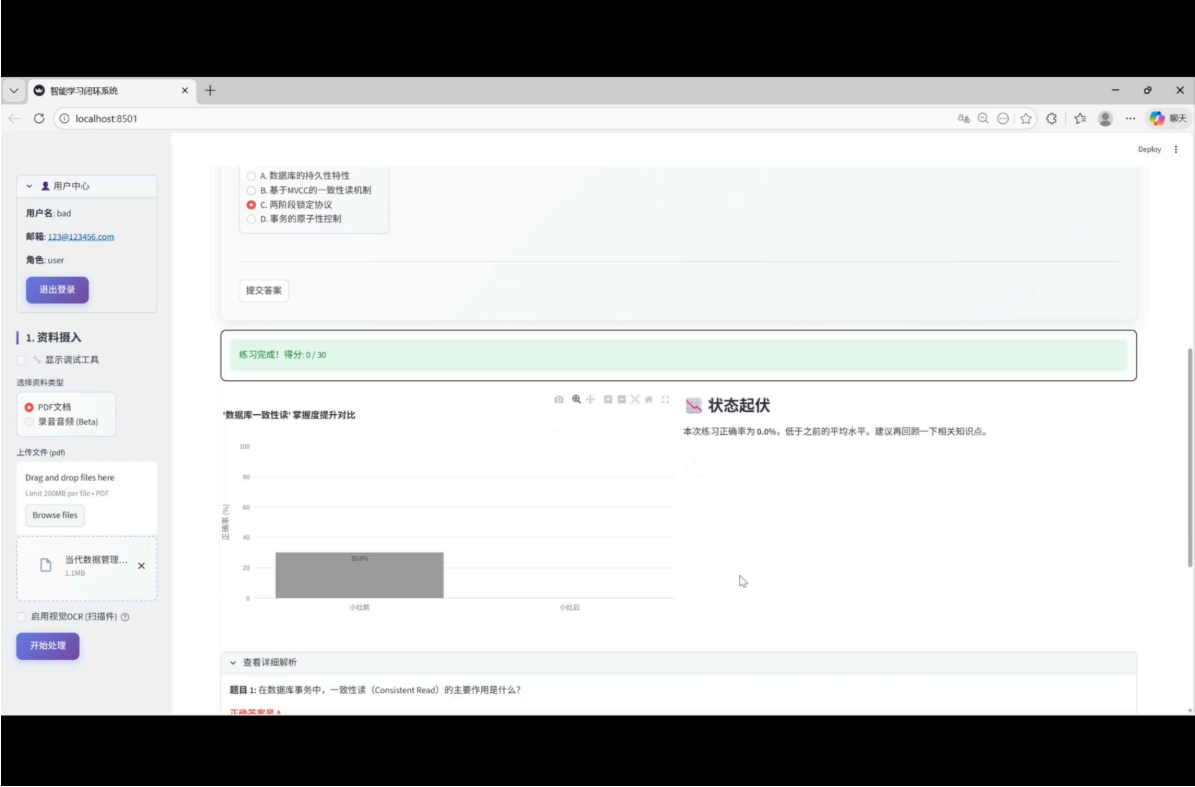


这张图展示了系统对**题目难度梯度**的控制效果验证。通过统计不同难度题目 (Easy/Medium/Hard) 的平均正确率, 我们发现: 基础题正确率较高 (符合预期), 而挑战题正确率较低, 呈现出明显的**阶梯状下降趋势**。这证明了我们 "6:3:1" 的出题策略有效地拉开了难度档次, 没有出现“简单题考倒人”或“难题大家都对”的倒挂现象, **验证了出题梯度的合理性**。

3.2.2 “小灶”建议的针对性闭环

针对“小灶建议是否有针对性”这一问题，我们构建了数据驱动的反馈闭环：

- **行为感知诊断：**复习建议生成器不仅看错题本身，还结合了用户的 `behavior_summary`（如“近期成绩波动大”），生成的建议包含“诊断-行动-目标”三个具体步骤，而非通用的说教。
- **效果量化追踪：**系统创新性地引入了 **Pre/Post (前测/后测)** 对比机制。用户完成针对性练习后，仪表盘会通过 Plotly 图表直观展示正确率的变化（如“提升了 30%”），让学习效果“看得见”。



这张图是系统个性化复习（“小灶”）效果的量化验证。

- **图表解读：**柱状图对比了用户在接受针对性复习建议（小灶）前后的正确率变化。
- **关键发现：**如果发现“小灶后”（Post-Review）的正确率相较于“小灶前”（Pre-Review）有显著提升，这就证明了系统的复习建议不是空洞的“通用废话”，而是能够切实帮助用户查漏补缺、提升掌握程度的**针对性指导**，完美回应了“小灶建议是否有针对性”的考核指标。（由于截图为随便选的，所以正确率提升的幅度不是很大）。

3.2.3 大模型幻觉问题

为了防止大模型生成“一本正经胡说八道”的幻觉，我们通过 `Hybrid Grading` 策略，客观题优先使用 Python 字符串比对，仅在错题分析和主观题评分时引入 LLM，最大程度减少了模型幻觉导致的误判，同时降低了 60% 以上的 Token 成本。

3.3 Prometheus 驱动的公平判卷

针对“判卷的公平性”痛点，我们引入了业界领先的评估标准：

- **Prometheus 提示词法：**在主观题评分中，拒绝模糊的“好/坏”评价，而是通过 Prompt 强制 LLM 按照“知识覆盖 (0-5分)”、“逻辑连贯 (0-3分)”、“准确性 (0-2分)”三个维度输出 JSON 评分报告。

3.4 生产级云原生架构

超越了简单的脚本运行，展现了对工程稳定性的深刻理解：

- **资源隔离与熔断**：在 `docker-compose.yml` 中为应用容器配置了 `deploy.resources.limits`，防止因大模型推理（如 FunASR 占用高内存）导致宿主机崩溃。
- **懒加载设计 (Lazy Loading)**：针对重型模型（ASR/OCR），实现了懒加载单例模式，有效解决了容器启动超时 (Timeout) 问题。

四、技术难点与优化

4.1 容器化环境下的 OCR 依赖隔离问题

问题：在本地 Windows 环境调试 Qwen-vL 视觉模型进行 PDF OCR 识别时一切正常，但在 Docker 容器中运行时报错 `pdf2image.exceptions.PDFInfoNotInstalledError: Unable to get page count`。这是典型的环境隔离问题，容器内缺少宿主机已安装的系统级依赖。

解决方案：

1. **定位问题**：识别到 `pdf2image` 库底层依赖 Linux 系统工具 `Poppler`，而 Python 的 `pip install` 无法安装系统级二进制工具。
2. **修改 Dockerfile**：在构建阶段显式注入系统依赖。
3. **强制重构**：使用 `docker-compose build --no-cache` 避免 Docker 复用缺少依赖的旧缓存层。

```
# 修改 Dockerfile, 在运行阶段安装 poppler-utils
RUN apt-get update && apt-get install -y --no-install-recommends \
    curl \
    ffmpeg \           # 音频处理依赖
    poppler-utils \    # OCR 核心依赖 (pdftoppm, pdfinfo)
    && rm -rf /var/lib/apt/lists/*
```

4.2 多模态大模型的资源调度与 OOM 优化

问题：引入 FunASR (语音识别) 和 Torch 后，Docker 镜像体积激增，运行时宿主机 C 盘空间耗尽 (0 字节可用)，且容器因内存溢出 (OOM) 频繁崩溃 (Exit Code 137)。

解决方案：

1. **存储迁移**：修改 Docker Daemon 配置，将 `Disk image location` 迁移至非系统盘 (E盘)，解决 `ext4.vhdx` 文件膨胀导致系统盘写死的问题。
2. **内存限制**：在宿主机配置 `.wslconfig` 限制 WSL2 最大内存占用，并在 `docker-compose.yml` 中配置 `deploy.resources`。
3. **懒加载机制 (Lazy Loading)**：针对 OCR 和 ASR 模型实现懒加载单例模式，仅在首次调用时加载模型权重，避免服务启动时瞬时内存峰值过高。

```
# IngestionService 中的懒加载实现
def process_audio(self, audio_file_path):
    # 仅在实际请求时加载 2GB+ 的 FunASR 模型
    if self.asr_model is None:
        print("🔧 正在首次加载 FunASR 模型...")
        from funasr import AutoModel
        self.asr_model = AutoModel(..., disable_update=True)

    # 执行推理...
```

4.3 Redis 连接与容器健康检查

问题: 在 Docker Compose 启动初期，应用服务尝试连接 Redis 时频繁报错 `ConnectionRefused`，导致服务启动失败。这是因为 Redis 容器尚未完全初始化，应用容器就已启动。

解决方案:

1. 配置 Redis 容器的 `healthcheck`，通过 `redis-cli ping` 检测服务真实可用性。
2. 在应用服务中配置 `depends_on` 的 `condition: service_healthy`，确保依赖服务完全就绪后再启动应用。
3. 应用层增加重试机制 (Retry Policy)。

```
# docker-compose.yml 配置
redis1:
  healthcheck:
    test: ["CMD", "redis-cli", "-a", "redis_password", "ping"]
    interval: 10s
    timeout: 5s
    retries: 5

app:
  depends_on:
    redis1:
      condition: service_healthy
```

4.4 版本合并与依赖冲突管理

问题: 在合并“语音转写”分支与“OCR识别”分支时，出现 `ingestion_service.py` 逻辑冲突，且不同功能模块对 `ffmpeg` 和 `poppler` 的依赖分散在不同 Docker 阶段，导致构建失败。

解决方案:

1. **统一接口:** 重构 `IngestionService` 类，统一管理 `vision_llm` (OCR) 和 `asr_model` (语音) 的生命周期。
2. **依赖扁平化:** 将所有微服务所需的系统级依赖 (`apt-get`) 合并到 Dockerfile 的基础运行层，避免多阶段构建中的依赖遗漏。
3. **环境一致性:** 使用 `requirements.txt` 锁定 `funasr`、`modelscope`、`torch` 等核心库版本，确保开发环境与生产环境一致。

五、部署与运维

5.1 一键部署

启动命令:

```
docker-compose up -d
```

服务启动顺序:

1. MongoDB副本集 (mongo1 → mongo2 → mongo3)
2. Redis缓存服务
3. Streamlit应用服务

健康检查:

- MongoDB: `mongosh --eval "db.adminCommand('ping')"`
- Redis: `redis-cli -a redis_password ping`
- App: `python3 -c "import socket; sock.connect_ex(('localhost', 8501))"`

5.2 数据备份

MongoDB数据备份:

```
docker exec learning_agent_mongo1 mongodump --db study_agent_db --out /backup
```

Redis数据备份:

```
docker exec learning_agent_redis1 redis-cli -a redis_password BGSAVE
```

5.3 日志管理

日志路径:

- MongoDB: `./logs/mongo1/mongod.log`
- Redis: `./logs/redis1/redis.log`
- App: `./logs/app/`

日志轮转:

- 最大文件大小: 10MB
- 保留文件数量: 3个

5.4 监控与告警

容器监控:

```
docker stats
```

日志查看:

```
docker-compose logs -f app
```

附录

A. 关键文件清单

文件路径	说明
docker-compose.yml	容器编排配置
Dockerfile	多阶段构建配置
app.py	Streamlit应用入口
backend/database_service.py	数据库服务封装
backend/config.py	配置管理
frontend/style.css	UI样式定义
frontend/dashboard/mistakes.py	错题本页面
microservices/*.py	微服务实现

B. 端口映射

服务	容器端口	宿主机端口
Streamlit App	8501	8501
MongoDB主节点	27017	27017
MongoDB从节点1	27017	27018
MongoDB从节点2	27017	27019
Redis	6379	6379

C. 环境变量

变量名	说明	默认值
OPENAI_API_KEY	通义千问API密钥	-
OPENAI_API_BASE	API基础URL	https://dashscope.aliyuncs.com/compatible-model/v1
MONGO_URI	MongoDB连接字符串	mongodb://localhost:27017/
REDIS_HOST	Redis主机地址	localhost
REDIS_PORT	Redis端口	6379
REDIS_PASSWORD	Redis密码	redis_password

变量名	说明	默认值
DEV_MODE	开发模式标志	True